

Exercício 03 , Parte B: Análise de Complexidade das Torres de Hanói

O algoritmo de Hanói é um dos exemplos mais clássicos de recursividade, a cada chamada ele divide o problema em dois subproblemas menores, o que gera uma quantidade de movimentos que dobra conforme n aumenta. Na prática isso significa crescimento exponencial, ou seja, $O(2^n)$. A tabela abaixo mostra os valores que o programa contou em execução, comparados com o que a fórmula $2^n - 1$ prevê:

n	Movimentos contados	$2^n - 1$ (esperado)
1	1	1
2	3	3
3	7	7
4	15	15
5	31	31
6	63	63

Os valores bate perfeitamente com a teoria, isso confirma que a complexidade é de fato exponencial. Vale lembrar que cada chamada recursiva empilha um novo frame na pilha de execução, então para n grande o próprio stack pode estourar antes mesmo de terminar os movimentos. Para $n = 20$ por exemplo seriam mais de 1 milhão de chamadas, o que já deixa bem claro porque esse tipo de algoritmo é considerado intratável na prática pra valores altos de n . O numero de movimentos contados pelo programa coincidiu exatamente com o esperado em todos os casos testados, o que também serve como uma espécie de validação de que a implementação recursiva está correta, se a função tivesse algum erro de caso base ou de passagem de parâmetros, os valores não bateriam.